

Building a Systematic Trading Bot — A Student Guide

A complete walkthrough of how a real 24/7 crypto trading bot is designed, using my CryptoBot as the worked example. The goal isn't to copy it — it's to teach you the principles so you can build the best kind of bot yourself.

Read this first — the honest disclaimer. This is an **educational** document, not financial advice. A trading bot is a piece of software that can lose real money fast. Every serious builder does this in order: **(1) paper-trade** (simulated money) until the logic is proven, **(2)** go live with an amount you can afford to lose entirely, **(3)** never trust a backtest as if it were the future. Markets are adversarial and mostly unpredictable. Most trading bots lose money. Build one to *learn systems design*, not to get rich.

1. What "systematic" actually means

There are two kinds of trading:

- **Discretionary** — a human looks at a chart and decides. Gut feel.
- **Systematic** — a set of rules decides. Same inputs → same decision, every time. No emotion, no "I have a feeling."

A bot is systematic by definition. That's its whole edge: it never gets scared, never gets greedy, never revenge-trades, and it can watch 30 coins at once, 24 hours a day. The flip side: **it only knows what you taught it**. A bad rule runs perfectly and loses money perfectly.

So the entire craft is: *design good rules, and prove they're good before risking money.*

2. The core loop (every bot is this loop)

Strip away the details and every trading bot is one loop:

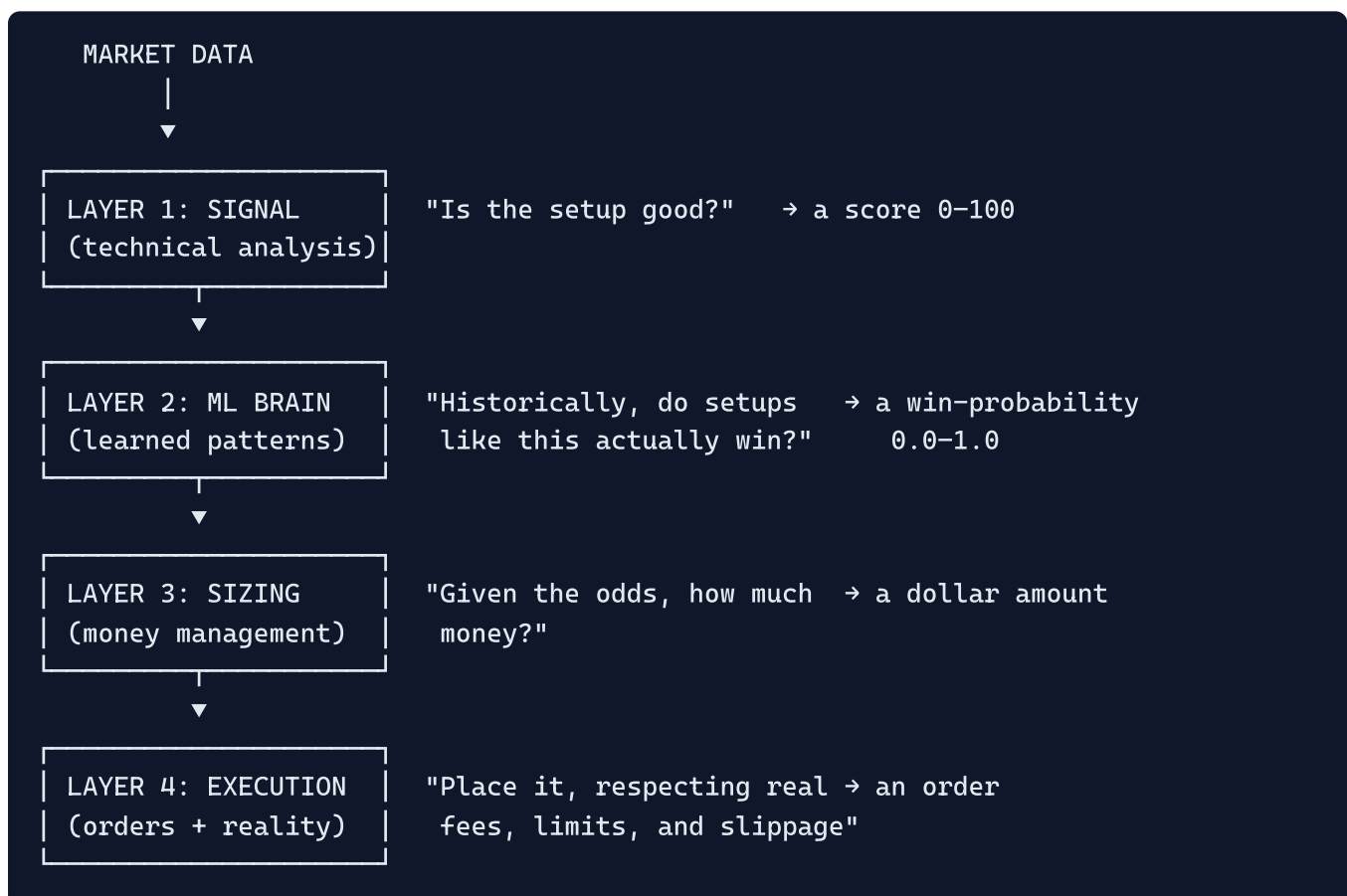
forever:

1. LOOK – pull fresh market data (prices, candles, volume)
 2. THINK – score each candidate: is this a good trade right now?
 3. SIZE – if yes, how much money should I put in?
 4. ACT – place the order
 5. MANAGE – watch open positions; exit when the plan says so
- sleep a few seconds, repeat

Everything else — indicators, machine learning, Kelly sizing — is just *making each of those five steps smarter*. If you understand the loop, you understand the bot. My CryptoBot runs this loop every 1–8 seconds depending on how aggressive it's set.

3. The 4-Layer Brain (separation of concerns)

The single most important design lesson: **don't tangle your logic into one giant function**. Split the decision into independent layers, each of which you can test and improve on its own.



Why this matters: when the bot loses money, you can ask *which layer was wrong*? Bad signal? Bad probability estimate? Over-sized? Bad fill? A monolith can't answer that. Layers can.

4. Layer 1 — Signal (technical analysis)

This layer turns raw candles into a single **score (0–100)** of how good a setup looks. My bot combines ~24 indicators. You do **not** need 24 — you need a few you understand. Here's the full menu, grouped by what they measure:

Trend (which way is price going?)

- **Moving Averages (SMA/EMA)** — is price above/below the average, and are the averages stacked bullishly? The 9/21 EMA cross is a classic.
- **SuperTrend** — an ATR-based line that flips green/red with the trend.
- **Ichimoku Cloud** — a full trend-and-support system in one.

Momentum (how strong is the move?)

- **RSI** — 0–100; below 30 = oversold, above 70 = overbought. Watch for *divergence* (price makes a new low, RSI doesn't → weakening downtrend).
- **MACD** — momentum via two moving averages; the histogram shows acceleration.
- **Stochastic**, **ADX** (trend strength), **TTM Squeeze** (volatility coiling before a breakout).

Volume / money flow (is real money behind it?)

- **OBV**, **Chaikin Money Flow**, **VWAP**, **relative volume**. Volume confirms moves — a breakout on thin volume is a trap.

Volatility (how much does it move?)

- **ATR (Average True Range)** — the single most useful number for a bot. It tells you how far this coin normally moves, so you can size stops and targets *proportionally*. More on this below — it's the key to per-coin tuning.
- **Bollinger Bands** — price relative to its recent volatility envelope.

Structure & patterns

- **Support/Resistance**, **Fibonacci retracements**, **swing highs/lows**.
- **Candlestick patterns** — Hammer, Bullish Engulfing, Morning Star, etc. (24 detected). Useful as *confirmation*, weak as standalone signals.

The lesson

Each indicator is a weak, noisy opinion. **Confluence** — several agreeing at once — is the signal. My bot weights each indicator's vote and sums them into the **scalp_score**. A trade only qualifies if that score clears a gate.

Multi-timeframe check: the top candidates get re-scored on a slower timeframe (15-minute) and the final score is $0.6 \times \text{fast} + 0.4 \times \text{slow}$. A setup that looks great on the 5-minute but terrible on the 15-minute is usually noise.

5. Layer 2 — The ML Brain (learning what actually works)

Here's the problem this layer solves: **indicators have known failure modes**. RSI divergence works in ranges and fails in strong trends. Candlestick patterns fire constantly and only sometimes matter. A human learns these exceptions from experience. A bot can too — from *data*.

My bot feeds every historical trade (the indicator readings at entry + whether it won or lost) into a machine-learning ensemble:

- **XGBoost + LightGBM** — two gradient-boosted decision-tree models. They learn which *combinations* of indicator readings actually preceded wins.
- **Calibration (isotonic regression)** — critical and often skipped. A raw model might output "0.9" for trades that only win 60% of the time. Calibration bends those outputs so that "0.7" genuinely means "wins ~70% of the time." You need *honest probabilities* for the sizing layer to work.
- **Walk-forward validation** — you train on the past and test on the *future* (data the model never saw), sliding forward through time. This is the only honest way to estimate a trading model. **Never** evaluate on data the model trained on — it will look brilliant and lose money live.

The lesson

ML is not magic and not required to start. Begin with pure indicator rules. Add ML only once you have a few hundred real trades to learn from — and treat it as a *filter on top of* your rules, not a replacement for understanding.

6. Layer 3 — Sizing (this is where bots live or die)

Most beginners obsess over entries and ignore sizing. That's backwards. You can be right 55% of the time and still go broke if you bet too big on the losers. Position sizing is risk management, and risk management is the whole game.

The Kelly Criterion

Kelly is a formula for the *optimal* bet size given your edge:

$$\text{fraction of bankroll} = \text{edge} / \text{odds}$$

Higher win-probability and bigger reward-to-risk → bigger bet. Lower → smaller. My bot runs the calibrated ML probability through Kelly to get a base size.

But full Kelly is dangerous — it's optimal for infinite bets with a known edge, and in real trading your edge estimate is always uncertain. So every serious system uses **fractional Kelly** (a fraction of what Kelly says) and **hard caps**. My bot layers on:

- **Risk-parity (inverse-ATR):** a calmer coin can hold a bigger position than a wild one for the same dollar risk. Size scales *down* as volatility scales up.
- **Sector concentration penalty:** don't put everything in one type of coin.
- **A hard per-position cap** (e.g. 20–40% of the account, by mode) that Kelly can never exceed, no matter how confident it is.

Goal-based throttling (protect gains, cap losses)

- Hit the daily profit target → cut size in half, raise the entry bar.
- Down 30% from the session peak → "guard mode": only A-grade setups.
- Down 40% → **hard stop**. The bot pauses itself. This "circuit breaker" is the most important line of code in the whole system.

The lesson

A good sizing layer turns a mediocre edge into a survivable strategy, and a bad one turns a real edge into a blow-up. Spend more time here than on indicators.

7. Layer 4 — Execution (respect reality)

The gap between backtest and live is *reality*: fees, slippage, minimum order sizes, exchange quirks. This layer handles them.

Fees change everything

Every round-trip trade pays a fee both ways. If a coin charges 0.25% per side, that's **0.5% round-trip** — so a +0.4% "winning" scalp is actually a *loss*. My bot is **fee-aware everywhere**: its profit targets are built as **price move needed = volatility + fees**, so it never takes a trade whose target can't clear the fee. One coin on the platform has **zero fees**, so the bot trades it far more aggressively (tiny +0.3% moves are profitable).

The lesson

A strategy that's profitable before fees and unprofitable after is the #1 way beginners fool themselves. Model fees from day one. Also model slippage (you rarely get the exact price you saw) and minimum order sizes.

8. Per-Coin Tuning (one size does NOT fit all)

This is the technique I'm proudest of, and it's a general principle: **don't use the same parameters for instruments that behave differently.**

Bitcoin normally moves ~1% in a session. A meme coin moves ~3%. If you put the same 1.5% stop-loss on both: - On Bitcoin, 1.5% is a real move — a good stop. - On the meme coin, 1.5% is *noise* — you get stopped out constantly on wiggles that mean nothing.

So the bot **derives each coin's parameters from its own volatility (ATR):**

```
edge_floor      = max(0.3%, 1.5 × round_trip_fee)  # never target below this
stop_loss       = scaled from 0.5×–1.5× the coin's ATR
take_profit     = scaled from 0.8×–3.0× the coin's ATR
harvest_target  = one ATR of move, after fees
min_hold_seconds = longer for wilder coins (don't whipsaw)
```

Same formula, different output per coin — Bitcoin gets tight stops and small targets; the meme coin gets wide ones. It **re-measures the volatility periodically** and re-tunes itself. This alone separates a toy bot from a real one.

9. Exit Management (the part beginners forget)

Everyone codes the entry. Almost nobody codes the exit properly, and **exits determine your P&L far more than entries.** My bot checks *every open position every second* against a ladder of exit rules, in priority order:

1. **Emergency stop** — down past a hard limit → get out now (overrides everything).
2. **Minimum hold** — don't react to noise for the first N seconds.
3. **Stop-loss** — price hit the pre-set risk level → exit.
4. **Take-profit** — price hit the target → exit.
5. **Trim (partial take-profit)** — at the first target, sell *part* (e.g. 40%) to bank profit, and let the rest run. "Take some off the table."
6. **Harvest** — lock a solid net gain quickly.
7. **Trail-lock / breakeven** — once a trade is up nicely, move the stop up so it can't turn into a loss; exit if it pulls back from its peak.
8. **Fade protection** — it peaked at +3% and faded to +1.8% → take what's left.
9. **Dump-bleed** — a slow leak that never hits the hard stop → cut it.

The two lessons

- **Cut losers fast, let winners run.** Every rule above is a version of this. The *trim* is especially powerful: it converts a paper gain into real money while keeping upside.
- **Exits are all-or-nothing OR partial.** A mature bot can do both — fully exit on a stop, but *trim* a winner. Beginners only code full exits.

10. The Learning Loop (get better over time)

After every closed trade, my bot updates three memories:

- **Per-coin reputation** — which coins it wins/loses on. Future sizing on a coin scales by its track record.
- **Per-indicator accuracy** — each indicator's vote is re-weighted (gently, $0.85\times-1.15\times$) based on how often it's been right lately. Only kicks in after 50+ trades, so it doesn't overreact to a small sample.
- **Full trade history** — feeds the ML retrain (every N trades).

The lesson

A bot that never learns runs the same mistakes forever. But **learn slowly** — overreacting to the last few trades ("it lost twice, disable that indicator!") is how bots destroy a real edge. Require a meaningful sample before you change behavior.

11. Risk Modes (one bot, many personalities)

Rather than hard-code one behavior, my bot exposes a single **mode** knob that swaps every threshold at once:

Parameter	Conservative	Regular	Sniper	Aggressive
Scan speed	8s	5s	3s	1s
Entry bar (gate)	very high	medium	premium only	loose
Max positions	3	6	1–2	30
Size per position	up to 20%	up to 20%	up to 40%	~6% (many small)
Profit target	+1.5%	+2%	+3%	+0.5%
Style	only A-grade	balanced	few big convictions	high-frequency

Same engine, four completely different strategies. This is good design: the *mechanism* is fixed, the *policy* is configurable.

12. Simulation First (the non-negotiable rule)

My bot runs a **paper-trading portfolio in parallel with the live one**, on the same signals. This does two things:

1. Lets you watch what the strategy *would* do with fake money before trusting it with real money.
2. Lets the bot try things it can't do live (e.g. short-selling) so you can study them safely.

Build the simulator first. Run it for weeks. Only go live when the paper results are consistently good — and even then, start tiny. A backtest can be overfit; forward paper-trading on live data is the honest test.

13. A checklist for building *your* best bot

If you take nothing else from this guide, take this order of operations:

1. **Pick one market and one timeframe.** Don't build for everything at once.
 2. **Get clean data** — reliable price/candle feed. Bad data = bad bot.
 3. **Code the loop** (look → think → size → act → manage). Keep the layers separate.
 4. **Start with 3–5 indicators you understand.** Not 24. Add later.
 5. **Model fees and slippage from day one.** Non-negotiable.
 6. **Size with fractional Kelly + a hard cap + a drawdown circuit breaker.** The circuit breaker is the most important code you'll write.
 7. **Write the exits before you go live.** Stop-loss, take-profit, trailing.
 8. **Paper-trade for weeks.** Judge it on forward results, not a backtest.
 9. **Go live with money you can lose entirely.** Start with the minimum.
 10. **Log everything and review.** A bot you can't inspect is a bot you can't fix.
 11. **Let it learn slowly.** Require a real sample before changing behavior.
 12. **Keep secrets out of your code.** API keys live in a config file or a secure input — never hard-coded, never committed to GitHub.
-

14. Project structure (how the code is organized)

A clean bot separates concerns into files. Here's the layout, as a template:

File	Responsibility
<code>app.py</code>	The main loop, the web dashboard, and orchestration
<code>analysis.py</code>	Layer 1 — indicators and the setup score
<code>indicators_pro.py</code>	Extra specialized indicators
<code>dip_detector.py</code>	A focused sub-strategy (buying oversold bounces)
<code>coin_strategies.py</code>	Per-coin parameter tuning + auto-tuner
<code>kelly_sizing.py</code>	Layer 3 — position sizing and risk
<code>ml_engine.py</code> / <code>ml_ensemble.py</code>	Layer 2 — the learned models
<code>learner.py</code>	The learning loop (adaptive weights + memory)
<code>portfolio_sim.py</code>	The paper-trading portfolio
<code>portfolio_live.py</code>	The live portfolio (talks to the exchange)
<code>exchange_client.py</code>	Layer 4 — the exchange adapter (orders, balance)
<code>screener.py</code>	Which coins to even look at (the universe)

Each file does one job. You can test, improve, or replace any one without breaking the others. **That modularity is the real lesson — it's what lets a bot grow from a toy into something that runs 24/7 for months.**

15. Final word

A trading bot is one of the best software projects you can build to learn real systems design: real-time data, decision pipelines, risk math, machine learning, state management, and dealing with an unforgiving external system (the market and the exchange). Build it for *that*. If it also makes money, treat that as a bonus you've earned by respecting risk — not as the goal that makes you skip the safety rails.

Paper-trade first. Cap your risk. Log everything. Learn slowly. Keep your keys secret. Do those five things and you're already ahead of most people who try this.

Happy building.